

Cognizant - 100 Technical Interview Questions

Cognizant Technology Solutions

With Answers & Diagrams - For BTech Computer Science / IT Students

Object-Oriented Programming

Q1. Explain the difference between 'is-a' and 'has-a' relationships with examples.

Answer:

An 'is-a' relationship implies inheritance - a Dog is-a Animal. A 'has-a' relationship implies composition - a Car has-a Engine. Choosing 'has-a' over 'is-a' when there isn't a true specialization relationship avoids incorrectly forcing an inheritance hierarchy where composition would be more appropriate.

Q2. What is operator overloading? Give an example in C++.

Answer:

Operator overloading lets you redefine what an operator like '+' does for a user-defined type. For example, in C++ you could overload '+' for a Complex number class so that `c1 + c2` adds their real and imaginary parts, letting Complex objects be used with familiar arithmetic syntax.

Q3. Explain the SOLID principles of object-oriented design.

Answer:

SOLID stands for: Single Responsibility (a class should have one reason to change), Open/Closed (open for extension, closed for modification), Liskov Substitution (subtypes must be substitutable for their base types without breaking correctness), Interface Segregation (prefer many small specific interfaces over one large one), and Dependency Inversion (depend on abstractions, not concrete implementations).

Q4. What is the Single Responsibility Principle, and why does violating it make code harder to maintain?

Answer:

The Single Responsibility Principle states a class should have only one reason to change - i.e., it should do one thing well. Violating it (e.g. a class that both parses files and sends emails) means unrelated changes end up modifying the same class, increasing the risk of breaking unrelated functionality and making the class harder to test and reuse.

Q5. What is object cloning, and what is the difference between a shallow copy and a deep copy?

Answer:

Object cloning creates a copy of an existing object. A shallow copy duplicates the object's top-level fields but nested objects/references are shared with the original (mutating a nested object affects both copies). A deep copy recursively duplicates nested objects too, so the copy is fully independent of the original.

Q6. Explain the concept of late binding vs early binding with an example.

Answer:

Early (static) binding resolves which method to call at compile time, based on the reference type - used for overloaded and static methods. Late (dynamic) binding resolves the method call at runtime, based on the actual object type - used for overridden instance methods, which is the basis of runtime polymorphism.

Q7. Explain what a static method is and why it cannot be overridden (only hidden) in Java.

Answer:

A static method belongs to the class itself rather than any instance, and is resolved at compile time based on the reference type, not the runtime object type. Since overriding relies on runtime (dynamic) dispatch, a static method with the same signature in a subclass merely 'hides' the parent's version rather than overriding it - the version called depends on which class name/reference type you use to call it.

Q8. Explain the purpose of the 'this' and 'super' keywords in Java.

Answer:

'this' refers to the current object instance, used to distinguish instance fields from parameters with the same name or to pass the current object elsewhere. 'super' refers to the immediate parent class, used to call a parent's constructor, method, or access a parent field that's been hidden by the subclass.

Q9. What happens when a class implements an interface but does not implement all its methods?

Answer:

If a class implements an interface but doesn't implement all its abstract methods, the class itself must be declared abstract; otherwise, it is a compile-time error since an interface is a contract that guarantees every method has an implementation somewhere in the concrete class hierarchy.

Q10. What are access modifiers? Explain public, private, protected, and default with examples.

Answer:

Access modifiers control visibility: 'public' - accessible from anywhere; 'private' - accessible only within the same class; 'protected' - accessible within the same package and by subclasses (even in different packages, in Java); 'default' (no modifier) - accessible only within the same package. They enforce encapsulation by controlling what other code can touch.

Q11. What is an object in OOP, and how is it different from a class?

Answer:

An object is a concrete instance of a class, with its own state (field values) stored in memory, while a class is the blueprint/template that defines what fields and methods its objects will have. 'Car' is the class; 'myRedCar' is an object of that class.

Q12. What is a functional interface in Java, and how does it relate to lambda expressions?

Answer:

A functional interface is an interface with exactly one abstract method (it may have default/static methods too), e.g. Runnable or Comparator. Java's lambda expressions are essentially shorthand implementations of a functional interface's single abstract method, letting you write (a, b) -> a.compareTo(b) instead of an anonymous inner class.

DBMS & SQL

Q13. Explain referential integrity and what happens when you try to delete a row referenced by a foreign key.

Answer:

Referential integrity ensures a foreign key value always points to an existing row in the referenced table (or is NULL). Trying to delete a row that is still referenced by a foreign key will fail with a constraint violation error, unless the foreign key was defined with ON DELETE CASCADE (deletes dependent rows too) or ON DELETE SET NULL (nulls out the reference) behavior.

Q14. Explain the difference between a foreign key constraint and a check constraint.

Answer:

A foreign key constraint enforces referential integrity - it ensures a column's value matches an existing value in the referenced table's primary/unique key (or is NULL), preventing orphaned references. A check constraint enforces a custom condition on the values allowed in a column (e.g. age >= 18), independent of any other table.

Q15. Write a SQL query using GROUP BY and HAVING to find departments with more than 5 employees.

Answer:

```
SELECT department, COUNT(*) AS emp_count FROM Employee GROUP BY department HAVING COUNT(*) > 5.
```

GROUP BY collapses rows into per-department groups, and HAVING filters those groups (after aggregation) to keep only departments with more than five employees.

Q16. What is sharding, and how does it help scale a database horizontally?

Answer:

Sharding splits a large table's rows horizontally across multiple database servers (shards), typically based on a shard key (e.g. customer_id range or hash), so no single machine needs to store or query the entire dataset. It improves write and storage scalability but introduces challenges like cross-shard joins/transactions and rebalancing when adding new shards.

Q17. Explain how you would design a database schema for a simple e-commerce order system (tables and relationships).

Answer:

A minimal e-commerce schema would include: Customers (customer_id PK, name, email), Products (product_id PK, name, price), Orders (order_id PK, customer_id FK, order_date, status), and OrderItems (order_item_id PK, order_id FK, product_id FK, quantity, price_at_purchase) as a junction table capturing the many-to-many relationship between Orders and Products.

Q18. Explain the difference between a primary key, a unique key, and a candidate key.

Answer:

A candidate key is any column (or set of columns) that could uniquely identify a row - a table can have several. A primary key is the one candidate key chosen to be the table's main unique identifier (cannot be NULL). A unique key is a constraint enforcing uniqueness on a column that is not the primary key, and unlike a primary key, it typically allows one NULL value.

Q19. Explain the difference between a correlated subquery and a regular subquery.

Answer:

A regular (uncorrelated) subquery executes once, independently of the outer query, and its result is used by the outer query. A correlated subquery references a column from the outer query and is conceptually re-evaluated for each row processed by the outer query, e.g. finding employees who earn more than the average salary of their own department.

Q20. What is a trigger, and give an example of when you would use one.

Answer:

A trigger is a stored procedure automatically executed in response to an INSERT, UPDATE, or DELETE event on a table. Example: a trigger that automatically logs the old and new values of a Salary column into an AuditLog table whenever an Employee row is updated, without requiring the application code to do it explicitly.

Q21. Explain the difference between a clustered and a non-clustered index.

Answer:

A clustered index determines the physical storage order of the table's data rows - a table can have only one, since data can only be sorted one way physically. A non-clustered index is a separate structure that stores pointers back to the actual data rows, so a table can have several, useful for speeding up queries on columns other than the clustering key.

Q22. What is database replication, and what is the difference between synchronous and asynchronous replication?

Answer:

Database replication maintains copies of data across multiple servers for redundancy and read scaling. Synchronous replication waits for the replica to confirm the write before the transaction commits (stronger consistency, higher write latency); asynchronous replication commits on the primary immediately and propagates changes to replicas afterward (lower latency, but a small risk of data loss if the primary fails before replicating).

Q23. What is a stored procedure, and what are its advantages over writing ad-hoc queries?

Answer:

A stored procedure is a precompiled, named collection of SQL statements stored in the database and invoked as a single call. Advantages include reduced network round-trips (one call instead of many statements), better security (grant execute rights without exposing underlying tables directly), reusability across applications, and a cached execution plan that can improve performance over repeatedly sent ad-hoc queries.

Q24. Explain optimistic vs pessimistic locking in database transactions.

Answer:

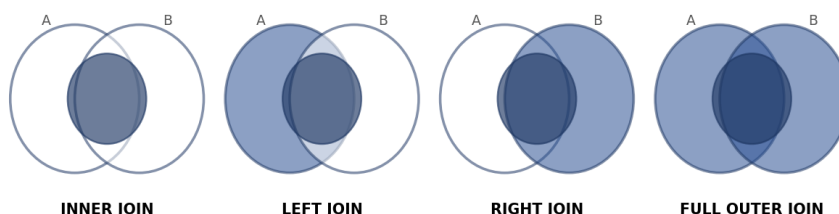
Pessimistic locking acquires a lock on a row before reading/modifying it, blocking other transactions until it's released - safe under high contention but can hurt throughput. Optimistic locking assumes conflicts are rare: it reads data without locking, then checks (e.g. via a version number) at commit time whether the data changed since it was read, retrying if a conflict is detected - better throughput when conflicts are infrequent.

Q25. Explain the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN with examples.

Answer:

INNER JOIN returns only rows with matching values in both tables. LEFT JOIN returns all rows from the left table plus matched rows from the right (NULLs where there's no match). RIGHT JOIN is the mirror - all rows from the right table plus matches from the left. FULL OUTER JOIN returns all rows from both tables, with NULLs wherever a match is missing on either side.

SQL JOIN Types (shaded = rows returned)



Q26. What is denormalization, and when would you intentionally denormalize a schema?

Answer:

Denormalization intentionally introduces redundancy (e.g. storing a customer's name directly on an Orders row instead of always joining to a Customers table) to reduce the number of joins needed and speed up read-heavy queries, at the cost of extra storage and more complex writes to keep the redundant copies in sync. It's used in reporting/analytics systems where read performance matters more than write simplicity.

Q27. Write a SQL query to find employees who earn more than the average salary of their department.

Answer:

`SELECT e.name, e.salary, e.department FROM Employee e WHERE e.salary > (SELECT AVG(salary) FROM Employee e2 WHERE e2.department = e.department).` The correlated subquery recomputes the department's average salary for each row and compares it to that employee's own salary.

Q28. What is the difference between OLTP and OLAP systems?

Answer:

OLTP (Online Transaction Processing) systems handle many short, frequent read/write transactions (e.g. an ATM system) and are optimized for fast inserts/updates on normalized schemas. OLAP (Online Analytical Processing) systems handle complex, read-heavy analytical queries over large historical datasets (e.g. a sales dashboard) and are optimized with denormalized/star schemas for fast aggregation.

Q29. How would you optimize a slow-running SQL query? Walk through your debugging approach.

Answer:

Approach: first run EXPLAIN/EXPLAIN ANALYZE to see the query plan and spot full table scans; check whether the filtered/joined columns have appropriate indexes; look for functions applied to indexed columns (which can prevent index use); check for unnecessary SELECT * pulling extra columns; consider rewriting correlated subqueries as joins; and check table statistics are up to date so the optimizer picks a good plan.

Q30. What is a view in SQL, and how is it different from a materialized view?

Answer:

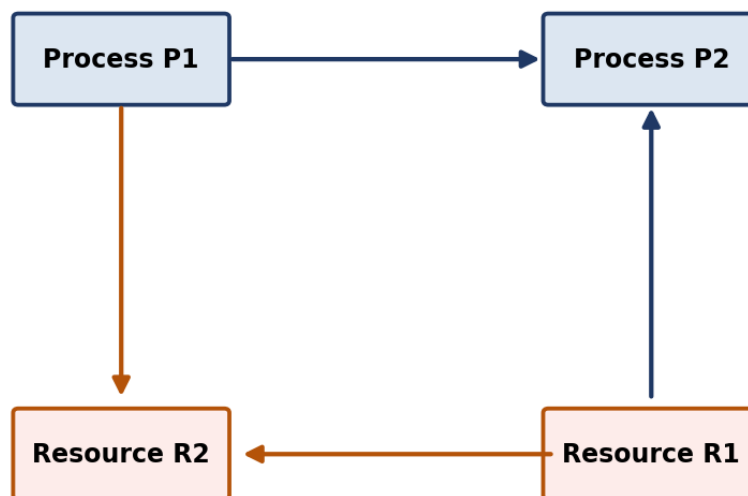
A view is a virtual table defined by a stored SELECT query - it has no data of its own and is recomputed each time it's queried. A materialized view physically stores the result set on disk and must be refreshed periodically (or on trigger), trading storage and staleness for much faster read access compared to recomputing a complex query every time.

Q31. What is a deadlock in a database, and how can it be prevented or resolved?

Answer:

A database deadlock happens when two or more transactions each hold a lock the other needs and wait on each other indefinitely (e.g. Transaction A locks Row 1 and waits for Row 2, while Transaction B locks Row 2 and waits for Row 1). It can be prevented by always acquiring locks in a consistent order across transactions, kept short by minimizing transaction scope, or resolved by the database's deadlock detector automatically rolling back one transaction (the 'victim') so the other can proceed.

Circular Wait: P1 -> R2 -> (held by P2) -> R1 -> (held by P1)



Q32. Explain database isolation levels and the anomalies (dirty read, non-repeatable read, phantom read) each one prevents.

Answer:

Isolation levels control how much one transaction can see of another's uncommitted work: Read Uncommitted allows dirty reads (seeing uncommitted changes); Read Committed prevents dirty reads but allows non-repeatable reads (a value changes between two reads in the same transaction); Repeatable Read prevents that but allows phantom reads (new rows appearing in a repeated range query); Serializable prevents all three by fully isolating transactions, at the cost of concurrency.

Q33. Write a SQL query to find duplicate records in a table.

Answer:

Using GROUP BY with HAVING COUNT(*) > 1: `SELECT column_name, COUNT(*) FROM table_name GROUP BY column_name HAVING COUNT(*) > 1`. This groups rows by the column(s) that define a duplicate and returns only the groups appearing more than once.

Q34. What is indexing, and what are the trade-offs of adding too many indexes to a table?

Answer:

An index is a data structure (commonly a B-Tree) that lets the database find rows matching a condition without scanning the whole table, dramatically speeding up SELECT queries. The trade-off is that every INSERT/UPDATE/DELETE must also update all indexes on that table, slowing down writes and consuming extra storage - so indexes should be added selectively on columns that are frequently filtered or joined on.

Operating Systems

Q35. What is a context switch, and why does it have a performance cost?

Answer:

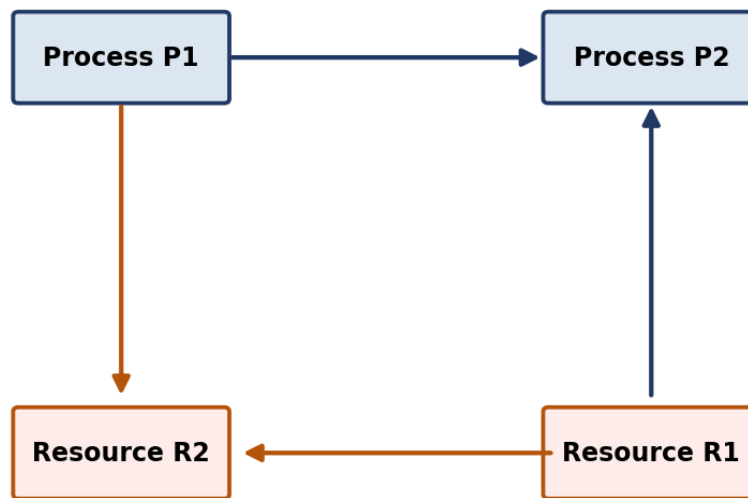
A context switch is the process of saving the CPU state (registers, program counter) of a currently running process/thread and loading the saved state of another, so execution can resume later exactly where it left off. It has a performance cost because the CPU does no useful work during the switch, and it can also cause cache/TLB misses afterward as the new process's data isn't cached yet.

Q36. What is a deadlock, and describe three general strategies for handling deadlocks (prevention, avoidance, detection & recovery).

Answer:

A deadlock is a state where a set of processes are permanently blocked, each waiting for a resource held by another in the set. Prevention negates one of the four necessary conditions upfront (e.g. requiring resources to be requested all at once). Avoidance checks each request dynamically for safety (e.g. Banker's Algorithm). Detection & Recovery lets deadlocks happen but periodically checks for cycles in the resource-allocation graph and recovers by killing/rolling back a process to break the cycle.

Circular Wait: P1 -> R2 -> (held by P2) -> R1 -> (held by P1)



Q37. What is the difference between a hard link and a symbolic link in a file system?

Answer:

A hard link is essentially another directory entry pointing directly to the same inode (and underlying data) as the original file - deleting the original name still leaves the data accessible via the hard link, and it cannot span file systems. A symbolic (soft) link is a small separate file that stores the path to the target file - it can point across file systems or to directories, but becomes a 'dangling link' if the target is deleted or moved.

Q38. Compare FCFS, SJF, Round Robin, and Priority scheduling algorithms - what are the trade-offs of each?

Answer:

FCFS (First-Come-First-Served) is simple but can cause the 'convoy effect' where a long job delays all others. SJF (Shortest Job First) minimizes average waiting time but requires knowing job lengths in advance and can starve long jobs. Round Robin gives each process a fixed time quantum, ensuring fairness and responsiveness for interactive systems, but too small a quantum increases context-switch overhead. Priority Scheduling runs the highest-priority job first but can starve low-priority jobs unless aging is used.

Q39. What is spooling, and how does it help manage I/O devices like printers?

Answer:

Spooling (Simultaneous Peripheral Operations On-Line) buffers data intended for a slow I/O device (like a printer) on disk, letting a process 'print' instantly to the spool without waiting for the physical device, while a separate spooler process feeds jobs to the device at its own pace - allowing multiple processes to share one device without conflict.

Q40. Explain how a page fault is handled by the operating system.

Answer:

When a process accesses a page not currently in physical memory, the CPU raises a page fault trap. The OS then: locates the required page on disk, finds a free frame (or selects a victim page to evict via a page replacement algorithm like LRU, writing it back to disk if modified), loads the required page into that frame, updates the page table, and resumes the instruction that caused the fault.

Q41. What is the difference between internal and external fragmentation?

Answer:

Internal fragmentation is wasted space inside an allocated fixed-size block that's larger than what the process actually needs (common in paging). External fragmentation is wasted space between allocated blocks - free memory exists in total but is scattered in pieces too small individually to satisfy a new request (common in segmentation/variable-size allocation).

Q42. What is a critical section, and what conditions must a solution to the critical-section problem satisfy?

Answer:

A critical section is a segment of code that accesses shared resources and must not be executed by more than one process/thread concurrently to avoid race conditions. A valid solution must guarantee: Mutual Exclusion (only one process in the critical section at a time), Progress (a process not in the critical section can't block others from entering), and Bounded Waiting (a limit on how many times other processes can enter before a waiting process gets its turn).

Q43. Explain the difference between a monolithic kernel and a microkernel.

Answer:

A monolithic kernel runs all core OS services (file system, device drivers, memory management, scheduling) in a single address space in kernel mode, which is fast (no IPC overhead between components) but a bug in any component can crash the whole system. A microkernel keeps only the bare minimum (IPC, basic scheduling, memory management) in kernel space and moves other services to user-space processes, improving fault isolation and modularity at the cost of more inter-process communication overhead.

Q44. What is a semaphore? Differentiate between binary and counting semaphores.

Answer:

A semaphore is an integer variable used for process synchronization, with atomic increment (signal/V) and decrement (wait/P) operations. A binary semaphore takes only values 0 or 1 and behaves like a mutex, ensuring exclusive access. A counting semaphore can take any non-negative integer, used to manage a pool of multiple identical resources (e.g. a fixed number of database connections).

Q45. Explain the concept of multithreading and its benefits and challenges.

Answer:

Multithreading lets multiple threads within one process run concurrently, sharing memory and resources. Benefits include better CPU utilization on multi-core systems, faster context switching between threads than processes, and easier data sharing. Challenges include race conditions, deadlocks, and the added complexity of correctly synchronizing access to shared data.

Q46. Explain how the Least Recently Used (LRU) page replacement algorithm works, and how you might implement it efficiently.

Answer:

LRU (Least Recently Used) evicts the page that hasn't been used for the longest time when a new page must be loaded and memory is full, based on the assumption that recently used pages are likely to be used again soon. An efficient implementation uses a combination of a hashmap (for $O(1)$ lookup of a page's location) and a doubly linked list ordered by recency, so both finding a page and moving it to the 'most recently used' end take $O(1)$ time.



$O(1)$ get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Computer Networks

Q47. Explain the difference between unicast, multicast, and broadcast communication.

Answer:

Unicast sends data from one sender to exactly one specific receiver (e.g. a normal web request). Multicast sends data from one sender to a specific group of interested receivers simultaneously (e.g. IPTV streaming to subscribed clients). Broadcast sends data from one sender to every device on the local network segment, regardless of interest (e.g. ARP requests, DHCP discovery).

Q48. What is the difference between a proxy server and a reverse proxy?

Answer:

A proxy server sits between clients and the internet, forwarding client requests outward - useful for caching, content filtering, or anonymizing client identity from the destination server's perspective. A reverse proxy sits in front of one or more backend servers, forwarding client requests to them - useful for load balancing, SSL termination, and hiding backend server details from clients.

Q49. Explain how SSL/TLS certificates establish trust between a client and a server.

Answer:

An SSL/TLS certificate, issued by a trusted Certificate Authority (CA), binds a public key to a domain's identity. During the TLS handshake, the server presents its certificate; the client verifies it was signed by a CA it trusts (via the certificate chain up to a trusted root CA), confirming the server is who it claims to be before establishing an encrypted session using keys derived during the handshake.

Q50. What is a CDN, and how does it reduce latency for end users?

Answer:

A CDN (Content Delivery Network) is a geographically distributed network of cache servers that store copies of static content (images, videos, scripts) closer to end users. When a user requests content, it's served from the nearest CDN edge server rather than the origin server, reducing round-trip latency and offloading traffic from the origin.

Q51. What is ARP, and how does it map an IP address to a MAC address?

Answer:

ARP (Address Resolution Protocol) resolves a known IP address to its corresponding MAC address on a local network. When a device needs to send a frame to an IP it doesn't have a MAC address for, it broadcasts an ARP request ('who has this IP?'), and the device owning that IP responds with its MAC address, which the sender then caches for future use.

Q52. What is the difference between a stateful and stateless protocol?

Answer:

A stateful protocol (like FTP or a typical database connection) requires the server to remember information about the client's session across multiple requests. A stateless protocol (like HTTP by default) treats every request independently, with no memory of previous requests - any state needed (like login status) must be carried explicitly in each request, e.g. via cookies or tokens.

Q53. Describe how you would troubleshoot a situation where a web application is reachable via IP but not via domain name.

Answer:

Troubleshooting approach: first verify DNS resolution actually works (nslookup/dig the domain) since reachability by IP but not domain strongly suggests a DNS issue - check the domain's A/AAAA/CNAME records are correctly configured and propagated, check the local machine's DNS resolver/cache and hosts file, and confirm the domain registrar/DNS provider settings point to the correct IP or load balancer.

Q54. Explain congestion control mechanisms used by TCP (e.g., slow start, congestion avoidance).

Answer:

TCP congestion control prevents a sender from overwhelming the network. 'Slow start' begins with a small congestion window and doubles it each round-trip until reaching a threshold or detecting loss. 'Congestion avoidance' then increases the window more conservatively (linearly) to probe for more bandwidth without causing congestion. On packet loss, TCP reduces the window sharply (e.g. halving it) and restarts the process, adapting to network conditions.

Data Structures & Algorithms

Q55. What is the difference between a greedy algorithm and dynamic programming? Give an example problem for each.

Answer:

A greedy algorithm makes the locally optimal choice at each step without reconsidering past decisions, and works only for problems where local optimality leads to a global optimum (e.g. activity/interval selection, Huffman coding). Dynamic programming considers overlapping subproblems and combines their optimal solutions, guaranteeing a global optimum for a broader class of problems (e.g. the 0/1 Knapsack problem, where greedy choice-by-choice doesn't always give the best result).

Q56. What is the sliding window technique, and in what type of problems is it useful?

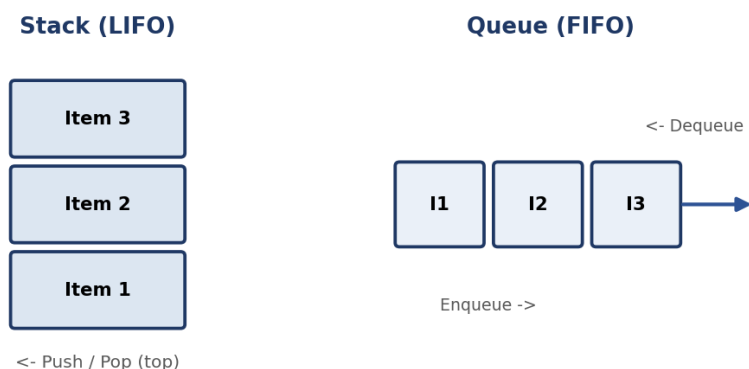
Answer:

The sliding window technique maintains a contiguous subrange ('window') over an array or string and expands/shrinks its boundaries incrementally rather than recomputing from scratch for every possible subrange. It's especially useful for problems asking for the best/longest/shortest subarray or substring satisfying some condition (e.g. maximum sum subarray of size k, or longest substring without repeating characters), typically turning an $O(n^2)$ brute force into an $O(n)$ solution.

Q57. What is the difference between a stack and a queue, and where is each used in real systems?

Answer:

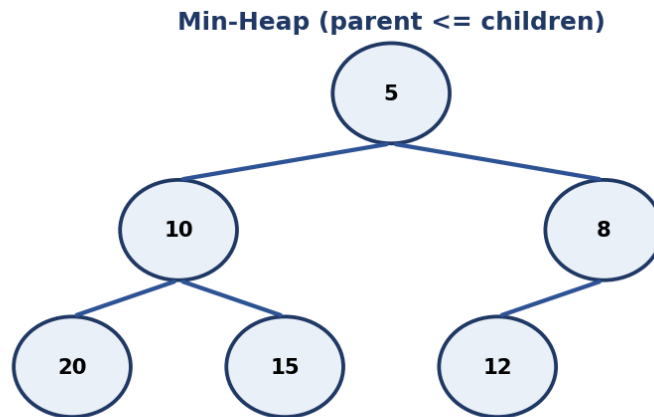
A stack follows Last-In-First-Out (LIFO) order - used for undo functionality, expression evaluation, and the call stack behind recursion. A queue follows First-In-First-Out (FIFO) order - used for task scheduling, breadth-first search, and handling requests in the order they arrive (e.g. print queues, message queues).



Q58. Explain how a min-heap/max-heap is implemented and how it supports efficient priority queue operations.

Answer:

A heap is a complete binary tree (usually implemented as an array) satisfying the heap property: in a min-heap, every parent is less than or equal to its children (so the minimum is always at the root); in a max-heap, every parent is greater than or equal to its children. Insertion and extraction of the min/max both take $O(\log n)$ since you only need to 'bubble' an element up or down the height of the tree, making heaps the standard backing structure for an efficient priority queue.



Q59. How would you find the kth largest element in an unsorted array efficiently?

Answer:

One efficient approach uses a min-heap of size k : iterate through the array, pushing elements onto the heap and popping the smallest whenever the heap exceeds size k - after processing all elements, the heap's root is the k th largest, achieving $O(n \log k)$ time. Alternatively, Quickselect (a partition-based approach related to Quicksort) finds it in average $O(n)$ time.

Q60. Explain topological sorting and where it is used.

Answer:

Topological sorting produces a linear ordering of a Directed Acyclic Graph's (DAG) vertices such that for every directed edge $u \rightarrow v$, u comes before v in the ordering. It's used wherever tasks have dependencies that must be respected, such as course prerequisite ordering, build systems compiling files in the right order, or task scheduling pipelines.

Q61. What is memoization, and how does it improve the performance of recursive algorithms?

Answer:

Memoization stores the result of a function call keyed by its input parameters (often in a hashmap or array), so if the same inputs occur again, the cached result is returned instantly instead of recomputing it. This turns algorithms with exponential-time naive recursion (like plain recursive Fibonacci) into polynomial-time algorithms by eliminating redundant recursive calls on the same subproblem.

Q62. Explain how a trie data structure works and where it is useful (e.g., autocomplete).

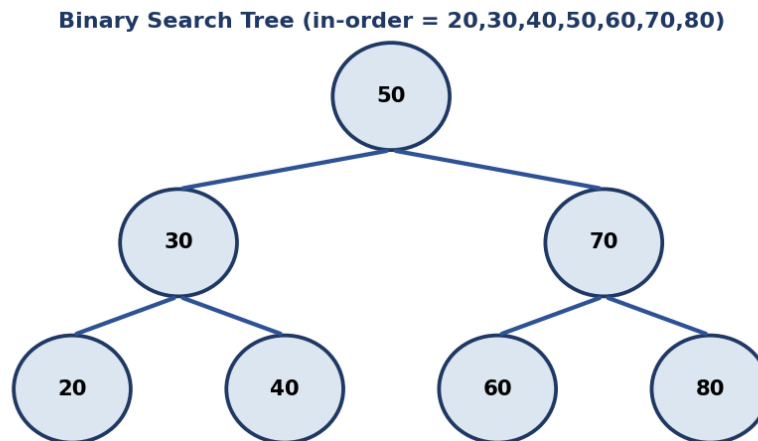
Answer:

A trie (prefix tree) stores strings character by character along tree edges, where each path from the root represents a prefix, and nodes are often marked to indicate a complete word ends there. It's especially useful for prefix-based operations like autocomplete/typeahead suggestions, spell-checking, and IP routing tables, since searching or inserting a string takes time proportional to the string's length rather than the number of stored strings.

Q63. What is the difference between a balanced and an unbalanced binary search tree, and why does balance matter?

Answer:

A balanced BST keeps its height close to $\log n$ by rebalancing after insertions/deletions (e.g. AVL trees, Red-Black trees), guaranteeing $O(\log n)$ search/insert/delete. An unbalanced BST can degenerate into something resembling a linked list if elements are inserted in sorted order without rebalancing, degrading all operations to $O(n)$. Balance matters because it's what actually delivers the logarithmic performance a BST is chosen for in the first place.



Q64. Explain the two-pointer technique with an example problem.

Answer:

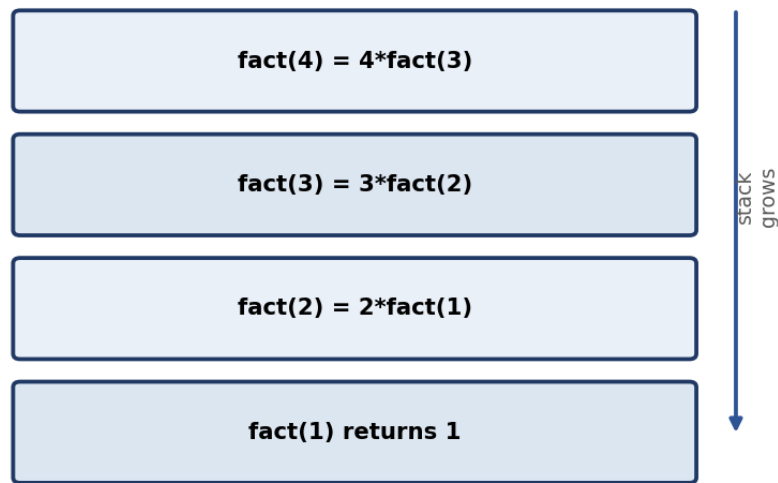
The two-pointer technique uses two indices moving through a data structure (often from opposite ends or at different speeds) to avoid nested loops. Example: given a sorted array, to find a pair summing to a target, start one pointer at the beginning and one at the end - if the sum is too large, move the right pointer left; if too small, move the left pointer right - solving the problem in $O(n)$ instead of the $O(n^2)$ brute force of checking every pair.

Q65. Explain how recursion uses the call stack internally, and what causes a stack overflow.

Answer:

Each recursive call pushes a new stack frame onto the call stack, containing that call's local variables, parameters, and return address. As calls nest deeper, more frames stack up; when a call returns, its frame is popped off. If recursion goes too deep (e.g. no base case, or a base case never reached due to a bug), the call stack exceeds its allocated size, causing a stack overflow.

Call Stack while computing factorial(4)



Q66. What is backtracking, and how is it different from brute force?

Answer:

Backtracking systematically explores all candidate solutions, but abandons ('backtracks' from) a partial candidate as soon as it determines that candidate cannot possibly lead to a valid solution, rather than exploring it to completion. This pruning makes it far more efficient than pure brute force, which would fully explore every possibility regardless of early signs of failure - classic examples include the N-Queens problem and Sudoku solvers.

Q67. Explain Dijkstra's algorithm for shortest paths and its limitations with negative edge weights.

Answer:

Dijkstra's algorithm finds the shortest path from a source node to all others in a weighted graph by repeatedly selecting the unvisited node with the smallest known distance, then 'relaxing' (updating) the distances of its neighbors - using a priority queue/min-heap, it runs in $O(E \log V)$. Its key limitation is that it assumes all edge weights are non-negative; a negative edge can invalidate its greedy assumption that once a node is finalized its distance can't improve, requiring an algorithm like Bellman-Ford instead.

Q68. Explain how you would design an LRU cache and the data structures you'd use to achieve $O(1)$ get/put operations.

Answer:

An LRU cache needs $O(1)$ get and put operations. The standard design combines a hashmap (key $\rightarrow$ pointer to a node) for instant lookup with a doubly linked list ordered by recency of use: on access, the accessed node is unlinked and moved to the 'most recently used' end in $O(1)$; when the cache exceeds capacity, the node at the 'least recently used' end is evicted, also in $O(1)$, with the hashmap updated accordingly.



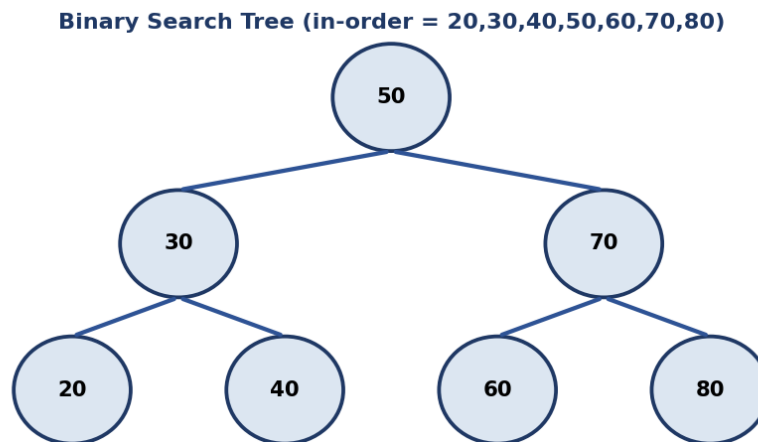
O(1) get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Coding Problems

Q69. Write a program to check if a given binary tree is a valid Binary Search Tree.

Answer:

Perform an in-order traversal of the tree and check that the resulting sequence of values is strictly increasing - a valid BST always produces a sorted sequence under in-order traversal. Alternatively, recursively validate each node against a running (min, max) valid range inherited from its ancestors, narrowing the range as you descend left or right - both approaches run in $O(n)$ time.



Q70. Write a program to implement binary search on a rotated sorted array.

Answer:

Modify binary search by first determining which half of the array (from mid to low, or mid to high) is properly sorted, then checking if the target lies within that sorted half's range - if so, search recurse/iterate into that half, otherwise search the other half. This preserves the $O(\log n)$ time complexity of standard binary search despite the rotation.

Q71. Write a program to check whether a given string is a palindrome.

Answer:

Use two pointers, one from the start (left) and one from the end (right) of the string, comparing characters and moving both pointers inward - if any pair doesn't match, it's not a palindrome; if the pointers meet or cross without mismatches, it is. This runs in $O(n)$ time and $O(1)$ extra space (ignoring the input itself).

Q72. Write a program to implement quicksort or mergesort from scratch.

Answer:

Quicksort: pick a pivot element, partition the array so elements smaller than the pivot come before it and larger elements come after, then recursively sort the two partitions - average $O(n \log n)$, worst case $O(n^2)$ with a poor pivot choice. Mergesort: recursively split the array in half until subarrays of size 1 remain, then repeatedly merge sorted subarrays back together - guaranteed $O(n \log n)$ in all cases, but needs $O(n)$ auxiliary space for merging.

Sorting Algorithm Complexity Comparison

Algorithm	Best	Average	Worst	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No

Q73. Write a program to merge overlapping intervals given a list of intervals.

Answer:

Sort the intervals by their start time, then iterate through them while maintaining a 'current merged interval' - if the next interval's start is less than or equal to the current merged interval's end, merge them by extending the end to the maximum of the two ends; otherwise, push the current merged interval to the result and start a new one with the next interval. This runs in $O(n \log n)$ time due to the initial sort.

Q74. Write a program to rotate an array by k positions.

Answer:

Reverse the whole array, then reverse the first k elements, then reverse the remaining n-k elements - this three-step reversal rotates the array by k positions in-place in $O(n)$ time and $O(1)$ extra space. (Alternatively, use an auxiliary array of size n to place each element directly at its rotated index, trading $O(n)$ space for simpler code.)

Q75. Write a program to check whether a binary tree is balanced.

Answer:

Recursively compute the height of the left and right subtrees for every node; if at any point the difference in height between left and right subtrees exceeds 1 for any node, the tree is unbalanced. An efficient implementation computes height and checks balance in the same bottom-up traversal (returning -1 as a sentinel to signal 'unbalanced' early), achieving $O(n)$ time instead of the naive $O(n^2)$.

Q76. Write a program to find the middle element of a linked list in a single pass.

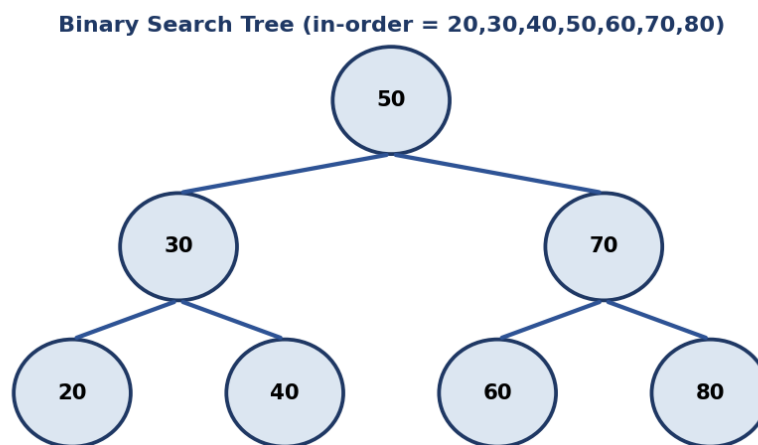
Answer:

Use the 'slow and fast pointer' technique: the slow pointer advances one node at a time while the fast pointer advances two nodes at a time - when the fast pointer reaches the end of the list, the slow pointer will be exactly at the middle. This finds the middle node in a single pass, in $O(n)$ time and $O(1)$ space.

Q77. Write a program to find the lowest common ancestor (LCA) of two nodes in a binary tree.

Answer:

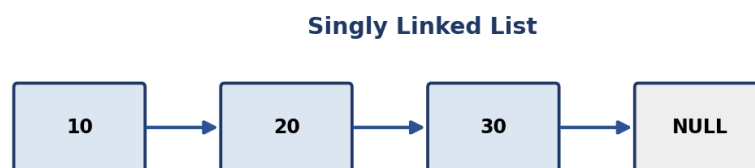
For a BST, the LCA can be found by starting at the root and comparing both target node values to the current node's value: if both are smaller, move to the left child; if both are larger, move to the right child; if they diverge (one smaller, one larger, or one equals the current node), the current node is the LCA. This runs in $O(h)$ time where h is the tree height. For a general binary tree (not necessarily a BST), a recursive post-order search that returns the first node where both targets are found in different subtrees is used instead.



Q78. Write a program to reverse a singly linked list (iterative and recursive approaches).

Answer:

Iterative: maintain a 'previous' pointer (initially null) and a 'current' pointer (initially head); at each step, save current.next, point current.next to previous, then advance previous and current forward - repeat until current is null, then previous is the new head. Recursive: recurse to the end of the list first, then reverse the pointer of each node on the way back up (node.next.next = node; node.next = null). Both run in $O(n)$ time; iterative uses $O(1)$ extra space while recursive uses $O(n)$ call-stack space.



Q79. Write a program to find the missing number in an array containing n distinct numbers from 0 to n.

Answer:

Sum formula approach: the expected sum of numbers 0 to n is $n*(n+1)/2$; subtract the actual sum of the given array from this expected sum, and the difference is the missing number - $O(n)$ time, $O(1)$ space. Alternatively, XOR every number from 0 to n together with every element in the array; since XOR-ing a number with itself cancels out to zero, all present numbers cancel, leaving only the missing number.

Q80. Write a program to find the longest common subsequence between two strings.

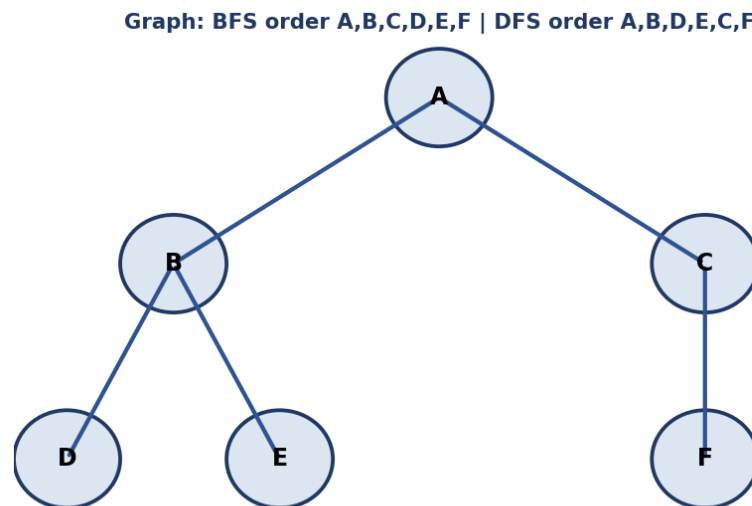
Answer:

Use dynamic programming: build a 2D table $dp[i][j]$ representing the length of the longest common subsequence between the first i characters of string A and first j characters of string B. If $A[i-1] == B[j-1]$, $dp[i][j] = dp[i-1][j-1] + 1$; otherwise $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$. This runs in $O(n*m)$ time and space, where n and m are the string lengths.

Q81. Write a program to count the number of islands in a 2D grid (graph/matrix traversal).

Answer:

Treat the grid as an implicit graph where each land cell ('1') is a node connected to its up/down/left/right land neighbors. Scan every cell; whenever an unvisited land cell is found, increment the island count and run a BFS or DFS from that cell to mark every connected land cell as visited, so it isn't counted again. This runs in $O(\text{rows} * \text{cols})$ time since each cell is visited a constant number of times.



Q82. Write a program to determine if a given number is prime, and optimize it for large inputs.

Answer:

Check divisibility only up to the square root of the number (if n has a factor larger than $\sqrt{n}$, it must also have a corresponding factor smaller than $\sqrt{n}$), skipping even numbers after checking 2, which reduces the check to roughly $O(\sqrt{n})$. For checking primality of many numbers up to a limit, the Sieve of Eratosthenes precomputes all primes up to that limit in $O(n \log \log n)$, which is far more efficient than testing each number individually.

Q83. Write a program to implement a basic LRU cache using a hashmap and a doubly linked list.

Answer:

Maintain a hashmap from key to a reference to its node in a doubly linked list ordered by recency of use. On `get(key)`: if present, move that node to the front (most recently used end) and return its value, else return `-1/None`. On `put(key, value)`: if the key exists, update its value and move it to the front; otherwise insert a new node at the front, and if capacity is exceeded, remove the node at the back of the list (least recently used) along with its hashmap entry. Both operations run in $O(1)$.



$O(1)$ get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Q84. Write a program to find all pairs in an array that sum up to a given target value.

Answer:

Use a hashmap: iterate through the array once, and for each element x , check if $(\text{target} - x)$ already exists in the hashmap - if so, record the pair; otherwise, add x to the hashmap and continue. This finds all pairs summing to the target in $O(n)$ time and $O(n)$ space, instead of the $O(n^2)$ brute-force nested loop.

Q85. Write a program to implement a queue using a fixed-size array (circular queue).

Answer:

A circular queue uses a fixed-size array with 'front' and 'rear' indices that wrap around using modulo arithmetic ($\text{rear} = (\text{rear} + 1) \% \text{capacity}$) once they reach the end of the array, allowing the queue to reuse freed space at the beginning after elements have been dequeued - avoiding the wasted space of a naive fixed-array queue that never reuses earlier slots.

Q86. Write a program to flatten a nested list/array.

Answer:

Use recursion: iterate through each element of the (possibly nested) list - if an element is itself a list, recursively flatten it and extend the result with its contents; if it's a plain value, append it directly to the result list. This naturally handles arbitrary nesting depth, with time complexity $O(n)$ where n is the total number of elements across all nesting levels.

Programming Language Fundamentals

Q87. What is the difference between '==' and '.equals()' when comparing objects in Java?

Answer:

'==' compares object references - whether two variables point to the exact same object in memory. '.equals()' compares the logical content/value of two objects, based on however the class has overridden it (e.g. String's .equals() compares character sequences). Two distinct String objects with the same text will be '==' false but '.equals()' true, unless string interning makes them the same reference.

Q88. Explain the difference between a shallow copy and a deep copy of a nested data structure.

Answer:

A shallow copy creates a new outer object but copies references to the same nested objects as the original - mutating a nested list inside a shallow copy also affects the original. A deep copy recursively creates independent copies of every nested object, so the copy is fully isolated from the original at every level, at the cost of more time and memory to perform the copy.

Q89. Explain Python's Global Interpreter Lock (GIL) and its impact on multithreaded CPU-bound programs.

Answer:

CPython's Global Interpreter Lock ensures only one thread executes Python bytecode at any given instant, even on a multi-core machine, to simplify memory management (reference counting) and avoid certain race conditions internally. This means CPU-bound multithreaded Python code doesn't actually get faster from adding more threads (they contend for the GIL) - for true parallel CPU-bound work, developers typically use multiprocessing (separate processes, each with its own interpreter and GIL) instead.

Q90. Explain the concept of multithreading vs multiprocessing, and when you would choose one over the other.

Answer:

Multithreading runs multiple threads within a single process, sharing memory - lightweight and fast to communicate between threads, but requires careful synchronization to avoid race conditions, and (in languages like Python with a GIL) doesn't achieve true CPU parallelism. Multiprocessing runs separate processes, each with its own memory space - avoids shared-memory race conditions and achieves true parallelism across CPU cores, but has higher overhead for creation and inter-process communication. Choose multithreading for I/O-bound work and multiprocessing for CPU-bound work in GIL-constrained languages.

Q91. What is a memory leak, and how would you identify one in a long-running application?

Answer:

A memory leak occurs when a program allocates memory but never releases it even though it's no longer needed, causing memory usage to grow over time and potentially exhausting available memory. In a long-running application, you'd identify one using a memory profiler to take heap snapshots over time and look for object counts/memory usage that keep growing without bound, then trace those objects back to what's still holding references to them (e.g. an ever-growing cache or a forgotten event-listener registration).

Q92. What is duck typing, and how does it relate to Python's dynamic type system?

Answer:

Duck typing means an object's suitability for an operation is determined by whether it has the necessary methods/attributes ('if it walks like a duck and quacks like a duck...') rather than its explicit declared type. Python's dynamic typing supports this naturally - a function can accept any object as long as it implements the methods the function actually calls, without requiring a shared base class or interface.

Q93. What is exception handling, and why is it important to avoid empty catch blocks?

Answer:

Exception handling is the mechanism for detecting and responding to runtime errors (like a missing file or invalid input) gracefully, using constructs like try/catch/finally, instead of letting the program crash. Empty catch blocks are dangerous because they silently swallow errors - the program continues running as if nothing went wrong, making it far harder to diagnose bugs later since there's no log, message, or fallback behavior indicating that something actually failed.

Q94. Explain the difference between an abstract class and a concrete class with an example from a language you know well.

Answer:

An abstract class can declare abstract methods (no body, must be implemented by subclasses) alongside concrete methods with actual implementations, and cannot be instantiated directly - e.g. an abstract Shape class with an abstract area() method but a concrete describe() method common to all shapes. A concrete class provides full implementations for all its methods and can be instantiated directly, e.g. a Circle class extending Shape and implementing area().

Q95. Explain the difference between static and dynamic memory allocation.

Answer:

Static memory allocation reserves a fixed amount of memory at compile time (e.g. a fixed-size array or a local variable on the stack), which is fast but inflexible since the size must be known ahead of time. Dynamic memory allocation reserves memory at runtime (e.g. via malloc/new), allowing the program to request exactly the amount of memory it needs based on runtime conditions, at the cost of manual (or garbage-collected) management and slightly slower access.

Q96. What are lambda expressions/anonymous functions, and when would you use them?

Answer:

A lambda expression (or anonymous function) is a small, unnamed function defined inline, typically used for short operations passed as arguments to other functions - e.g. sorting a list with a custom key: sorted(items, key=lambda x: x.age). They're best used for simple, one-off logic; anything more complex or reused in multiple places is usually clearer as a named function.

Web & Cloud Fundamentals

Q97. What is auto-scaling in cloud computing, and what metrics typically trigger it?

Answer:

Auto-scaling automatically adjusts the number of running compute instances up or down based on real-time demand, rather than requiring manual intervention. Common triggering metrics include CPU utilization, memory usage, request/queue length, and response latency - when a threshold is crossed for a sustained period, the auto-scaler adds or removes instances accordingly.

Q98. Explain the difference between authentication and authorization, and how OAuth 2.0 fits in.

Answer:

Authentication verifies who a user is (e.g. checking a username/password or token), while authorization determines what an authenticated user is allowed to do (e.g. which resources/actions they can access). OAuth 2.0 is an authorization framework that lets a user grant a third-party application limited access to their resources on another service (e.g. 'Sign in with Google') without sharing their actual password, typically issuing access tokens that represent specific granted permissions.

Q99. Explain the concept of statelessness in REST APIs and why it matters for scalability.

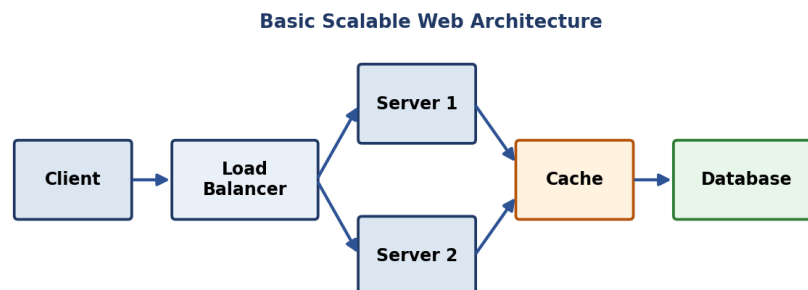
Answer:

Statelessness means each request from a client to a server must contain all the information the server needs to process it, with no reliance on server-side session state stored between requests. This matters for scalability because any server instance can handle any request without needing shared session state, making it trivial to add more server instances behind a load balancer without worrying about which server 'remembers' a given client.

Q100. What is the purpose of an API Gateway in a microservices-based application?

Answer:

An API Gateway sits in front of a microservices architecture as the single entry point for client requests, routing each request to the appropriate backend service. It commonly also handles cross-cutting concerns centrally - authentication/authorization, rate limiting, request/response transformation, and logging - so individual microservices don't each need to reimplement that logic.



Note: 16 of these 100 questions include a supporting diagram. Content compiled from publicly available 2026 placement-prep resources and established CS-fundamentals question banks - verify current-year specifics before your interview. Practice explaining answers out loud or in writing, not just reading - recall under pressure is the real test.